
API Security Risk Analysis Report

Read-Only Security Assessment

Date: April 12, 2026

Analyst: Security Assessment Team

Classification: Public

Target APIs	JSONPlaceholder & ReqRes
Assessment Type	Read-Only / Non-Intrusive
Methodology	OWASP API Security Top 10 (2023)
Tools Used	cURL, Browser DevTools

Table of Contents

- 1. Executive Summary
- 2. Scope & Methodology
- 3. APIs Tested
- 4. Findings Summary
- 5. Detailed Findings
- 6. Risk Matrix
- 7. Remediation Recommendations
- 8. Conclusion
- 9. References

1. Executive Summary

This report presents the results of a read-only API security risk analysis conducted on two publicly available demo APIs: **JSONPlaceholder** and **ReqRes**. The assessment followed the OWASP API Security Top 10 (2023) methodology, focusing on identifying common security risks without any exploitation or intrusive testing.

JSONPlaceholder, a widely used fake REST API for testing, exhibited several security patterns that would be critical vulnerabilities in a production environment, including completely unauthenticated CRUD operations and excessive data exposure. ReqRes demonstrated stronger security posture by requiring API key authentication on all endpoints.

Key Finding: JSONPlaceholder exposes all endpoints without any authentication, allows write/delete operations without authorization, and returns excessive personally identifiable information (PII) in API responses. While acceptable for a demo API, these patterns represent **HIGH** risk if replicated in production systems.

Metric	Value
Total Findings	6
High Severity	3
Medium Severity	2
Low Severity	1
APIs Tested	2

2. Scope & Methodology

2.1 Scope

This assessment was limited to read-only, non-intrusive analysis of publicly available demo APIs. No exploitation, bypass attempts, or denial-of-service testing was performed.

- â Read-only HTTP requests (GET, safe POST)
- â Header and response inspection
- â Authentication mechanism review
- â Documentation-based analysis
- â Response data exposure assessment

2.2 Methodology

The assessment followed the **OWASP API Security Top 10 (2023)** framework, covering:

- â API1: Broken Object Level Authorization (BOLA)
- â API2: Broken Authentication
- â API3: Broken Object Property Level Authorization
- â API4: Unrestricted Resource Consumption
- â API5: Broken Function Level Authorization
- â API7: Server-Side Request Forgery

2.3 Tools

Tool	Purpose
cURL	HTTP request testing and header inspection
Browser DevTools	Network traffic and response analysis
Manual Review	Documentation and endpoint analysis

3. APIs Tested

3.1 JSONPlaceholder

URL: <https://jsonplaceholder.typicode.com>

A free online REST API for testing and prototyping. Provides fake data for /posts, /users, /comments, /albums, /photos, and /todos endpoints. Supports full CRUD operations (GET, POST, PUT, PATCH, DELETE) without authentication.

3.2 ReqRes

URL: <https://reqres.in>

An API testing platform that simulates real API behavior. As of 2026, ReqRes requires an x-api-key header for all endpoints, returning 401 Unauthorized without it. The API also implements robust security headers including HSTS, X-Frame-Options, and Content Security Policy.

4. Findings Summary

#	Finding	API	Severity	OWASP Ref
F-01	No Authentication on Any Endpoint	JSONPlaceholder	HIGH	API2
F-02	Excessive Data Exposure (PII)	JSONPlaceholder	HIGH	API3
F-03	Unrestricted Write/Delete Operations	JSONPlaceholder	HIGH	API5
F-04	Missing Security Headers	JSONPlaceholder	MEDIUM	—
F-05	Overly Permissive CORS Policy	Both	MEDIUM	—
F-06	Server Technology Disclosure	JSONPlaceholder	LOW	—

5. Detailed Findings

F-01: No Authentication on Any Endpoint

Severity: HIGH

API: JSONPlaceholder

OWASP Reference: API2: Broken Authentication

Description: All JSONPlaceholder endpoints are accessible without any form of authentication. GET, POST, PUT, PATCH, and DELETE requests all succeed without tokens, API keys, or session cookies. Any anonymous user can read all data and simulate write operations.

Evidence: cURL requests to /users, /posts, and /comments all returned 200 OK without any Authorization header. POST to /posts returned 201 Created. DELETE to /posts/1 returned 200 OK.

Impact: In a production scenario, this would allow unauthorized access to all data and operations, enabling data theft, manipulation, and deletion by any party.

Remediation: Implement OAuth 2.0 or JWT-based authentication. Require valid tokens for all endpoints. Use API keys for public-facing read endpoints at minimum.

F-02: Excessive Data Exposure (PII in Responses)

Severity: HIGH

API: JSONPlaceholder

OWASP Reference: API3: Broken Object Property Level Authorization

Description: The /users endpoint returns extensive personally identifiable information including full name, email address, phone number, physical address (street, suite, city, zipcode), GPS coordinates, and company details in every response.

Evidence: GET /users/1 returned: name, username, email (Sincere@april.biz), phone (1-770-736-8031 x56442), full address with geo coordinates (lat: -37.3159, lng: 81.1496).

Impact: Excessive data in API responses increases the attack surface. If replicated in production, this could violate GDPR, CCPA, and other data protection regulations. Location data exposure is particularly sensitive.

Remediation: Implement field-level filtering. Return only necessary fields per endpoint. Use separate authenticated endpoints for sensitive PII. Apply data minimization principles.

F-03: Unrestricted Write and Delete Operations

Severity: HIGH

API: JSONPlaceholder

OWASP Reference: API5: Broken Function Level Authorization

Description: All CRUD operations (Create, Read, Update, Delete) are available to unauthenticated users. There is no role-based access control or authorization checks. Any user can create, modify, or delete any resource.

Evidence: POST /posts with arbitrary body returned 201 Created with id: 101. DELETE /posts/1 returned 200 OK. No authorization headers required for either operation.

Impact: In production, this would allow mass data manipulation or deletion by any party, data integrity compromise, and potential service disruption.

Remediation: Implement RBAC (Role-Based Access Control). Separate read/write permissions. Require elevated privileges for DELETE operations. Add ownership validation for updates.

F-04: Missing Security Headers

Severity: MEDIUM

API: JSONPlaceholder

Description: JSONPlaceholder response headers lack several security-critical headers that are present in ReqRes. Missing headers include: Strict-Transport-Security (HSTS), X-Frame-Options, Referrer-Policy, and Cross-Origin-Resource-Policy.

Evidence: Header inspection showed only x-content-type-options: nosniff and access-control-allow-credentials: true. No HSTS, X-Frame-Options, or CSP headers.

Impact: Missing HSTS allows potential SSL stripping attacks. Missing X-Frame-Options enables clickjacking. Missing Referrer-Policy may leak sensitive URL parameters.

Remediation: Add: Strict-Transport-Security: max-age=31536000; includeSubDomains. X-Frame-Options: DENY. Referrer-Policy: strict-origin-when-cross-origin. Content-Security-Policy with appropriate directives.

F-05: Overly Permissive CORS Policy

Severity: MEDIUM

API: Both APIs

Description: Both APIs use permissive CORS configurations. JSONPlaceholder sets access-control-allow-credentials: true without restricting origins. ReqRes sets access-control-allow-origin: *. These configurations could allow cross-origin attacks in production contexts.

Evidence: JSONPlaceholder: access-control-allow-credentials: true (allows cookies from any origin). ReqRes: access-control-allow-origin: * (allows requests from any domain).

Impact: Overly permissive CORS can enable cross-site data theft if combined with authentication vulnerabilities. Wildcard origins with credentials enabled is particularly dangerous.

Remediation: Restrict Access-Control-Allow-Origin to specific trusted domains. Never combine wildcard origins with Allow-Credentials: true. Implement origin validation on the server side.

F-06: Server Technology Disclosure

Severity: LOW

API: JSONPlaceholder

Description: Response headers reveal the server technology stack, including X-Powered-By: Express and server: cloudflare. This information helps attackers identify known vulnerabilities specific to these technologies.

Evidence: Headers: x-powered-by: Express, server: cloudflare, via: 2.0 heroku-router.

Impact: Technology disclosure enables targeted attacks against known Express.js or Node.js vulnerabilities. Low severity as a standalone issue but compounds with other findings.

Remediation: Remove X-Powered-By header (`app.disable('x-powered-by')` in Express). Consider removing or obfuscating server headers where possible.

6. Risk Matrix

6.1 Comparative Security Posture

Security Control	JSONPlaceholder	ReqRes
Authentication Required	No	Yes
HSTS Enabled	No	Yes
X-Frame-Options	Missing	DENY
Rate Limiting Headers	Yes	Not Observed
X-Content-Type-Options	Yes	Yes
CORS Restriction	Permissive	Wildcard
Technology Disclosure	Yes â■■■	Minimal

7. Remediation Recommendations

7.1 Priority Actions (Immediate)

- â **Implement Authentication:** Deploy OAuth 2.0 / JWT for all endpoints. No API should be publicly writable without credentials.
- â **Apply Data Minimization:** Remove PII from default responses. Use field selection parameters. Never expose geo-coordinates by default.
- â **Enforce Authorization:** Implement RBAC. Separate read/write roles. Validate resource ownership before mutations.

7.2 Short-Term Improvements

- â **Add Security Headers:** HSTS, X-Frame-Options: DENY, CSP, Referrer-Policy on all responses.
- â **Restrict CORS:** Replace wildcard origins with explicit domain allowlists. Never combine credentials with wildcard.
- â **Remove Technology Disclosure:** Strip X-Powered-By and minimize server header information.

7.3 Long-Term Strategy

- â Implement API gateway with centralized auth, rate limiting, and monitoring
- â Deploy Web Application Firewall (WAF) with API-specific rulesets
- â Establish regular API security audit cadence (quarterly)
- â Implement API versioning and deprecation policies
- â Add request/response logging for audit trails

8. Conclusion

This analysis demonstrates the critical importance of API security in modern applications. JSONPlaceholder, while designed as a testing tool, exhibits security patterns that are commonly found in production APIs — particularly the lack of authentication, excessive data exposure, and unrestricted write access.

ReqRes demonstrates a significantly better security posture by requiring API key authentication and implementing comprehensive security headers. This comparison highlights how straightforward security controls can drastically reduce an API's attack surface.

Organizations should treat API security as a first-class concern, implementing authentication, authorization, data minimization, and proper header configuration as baseline requirements for any API deployment.

9. References

â─ OWASP API Security Top 10 (2023) — <https://github.com/OWASP/API-Security>

â─ API Security Checklist — <https://github.com/shieldfy/API-Security-Checklist>

â─ JSONPlaceholder — <https://jsonplaceholder.typicode.com>

â─ ReqRes — <https://reqres.in>

â─ NIST SP 800-95: Guide to Secure Web Services

â─ RFC 6749: OAuth 2.0 Authorization Framework

This report was generated as part of an ethical, read-only API security assessment. No systems were exploited or harmed during this analysis.

© 2026 Security Assessment Team. All rights reserved.